# Apprentissage Automatique Simple

## Mathématiques et Implémentation Python

Sitraka FORLER

Centrale Marseille

*Machine Learning - Novembre 2025*

20 mars 2026

# Table des Matières


1. Introduction à l'Apprentissage Automatique
2. Régression Linéaire
3. Classification Logistique
4. Régularisation
5. Avec Scikit-learn
6. Concepts Avancés
7. Bonnes Pratiques

# Qu'est-ce que l'Apprentissage Automatique ?

**Définition :** L'apprentissage automatique est la science de permettre aux ordinateurs d'apprendre des données sans être explicitement programmés.

**Trois types principaux :**

- **Apprentissage Supervisé** : données labelisées (entrée $\rightarrow$ sortie)
- **Apprentissage Non-supervisé** : données non labelisées (découverte de patterns)
- **Apprentissage par Renforcement** : apprentissage par récompenses

**Ce cours couvre :** Apprentissage Supervisé (Régression et Classification)

# Concepts Clés

**Données :**

- **Features (X)** : variables d'entrée
- **Target (y)** : variable à prédire
- **m** : nombre d'exemples
- **n** : nombre de features

**Modèle :**

- **Paramètres $\theta$** : poids à apprendre
- **Prédiction** : $\hat{y} = f(X, \theta)$
- **Coût** : mesure d'erreur
- **Optimisation** : minimiser la perte

**Processus :**

Données $\rightarrow$ Modèle $\rightarrow$ Prédictions $\rightarrow$ Coût $\rightarrow$ Meilleurs paramètres

# Régression Linéaire : Concept

**Objectif :** Prédire une valeur continue (prix, température, etc.)

**Hypothèse du modèle :**

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^T \mathbf{x}$$

**Forme matricielle :**

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$

Où $\mathbf{X}$ est la matrice de design avec colonne de 1 pour le biais.

**Erreur Quadratique Moyenne (MSE) :**

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^{m} (y^{(i)} - \hat{y}^{(i)})^2$$

# Équation Normale

**Solution directe (sans itération) :**

$$\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

**Implémentation Python :**

```
import numpy as np

def normal_equation(X, y):
    """Calcule les coefficients par  quation  normale
       """
    theta = np.linalg.inv(X.T @ X) @ X.T @ y
    return theta

# Exemple d'utilisation
X = np.array([[1, 100], [1, 150], [1, 200]])
y = np.array([200, 250, 300])
theta = normal_equation(X, y)
print(f"Coefficients: {theta}")
```

**Avantages :**

# Gradient Descent

**Mise à jour itérative :**

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

**Implémentation :**

```
def gradient_descent(X, y, alpha=0.001, iterations
    =1000):
    m, n = X.shape
    theta = np.zeros(n)

    for i in range(iterations):
        y_pred = X @ theta
        gradient = (1/m) * X.T @ (y_pred - y)
        theta = theta - alpha * gradient

        if i % 100 == 0:
            cost = np.mean((y_pred - y)**2) / 2
            print(f"It ration {i}: Co t = {cost:.4f}
                ")
```

# Équation Normale vs Gradient Descent

| Critère | Équation Normale | Gradient Descent |
|---|---|---|
| Itérations | 0 (directe) | 100-10000 |
| Complexité | $O(n^3)$ | $O(n \cdot \text{iter})$ |
| Bon pour | Petit $n$ | Grand $n$ |
| Streaming | Non | Oui |

Table – Comparaison des deux approches

**Recommandation :**

- **Petit** $n$ : Équation Normale (plus rapide)
- **Grand** $n$ : Gradient Descent (efficace mémoire)
- **Données streaming** : Gradient Descent

# Classification Logistique

**Objectif :** Prédire une classe binaire (oui/non, spam/non-spam)

**Solution :** Fonction Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

**Modèle complet :**

$$P(y = 1|x) = \sigma(\boldsymbol{\theta}^T \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}$$

**Propriétés clés :**

- Sortie toujours entre 0 et 1
- $\sigma(0) = 0.5$ (point de décision)
- Dérivée : $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

# Fonction Sigmoid

**Implémentation :**

```
def sigmoid(z):
    """Fonction sigmoid"""
    return 1 / (1 + np.exp(-z))

# Test
z_values = np.array([-2, -1, 0, 1, 2])
probs = sigmoid(z_values)

for z, p in zip(z_values, probs):
    print(f"z={z:.1f}        (z)={p:.3f}")

# Visualisation
import matplotlib.pyplot as plt
z = np.linspace(-5, 5, 100)
plt.plot(z, sigmoid(z))
plt.axhline(y=0.5, color='r', linestyle='--')
plt.grid()
plt.show()
```

# Entropie Croisée Binaire

**Fonction de coût pour classification :**

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

**Implémentation complète :**

```
def logistic_regression(X, y, alpha=0.01,
                        iterations=1000):
    m, n = X.shape
    theta = np.zeros(n)

    for i in range(iterations):
        z = X @ theta
        y_pred = sigmoid(z)
        gradient = (1/m) * X.T @ (y_pred - y)
        theta = theta - alpha * gradient

    return theta
```

# Évaluation : Métriques de Classification

```python
def evaluate_classification(y_true, y_pred):
    """Calcule les m triques de classification"""
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))

    accuracy = (TP + TN) / (TP + TN + FP + FN)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    f1 = 2 * (precision * recall) / (precision +
        recall)

    print(f"Accuracy: {accuracy:.3f}")
    print(f"Precision: {precision:.3f}")
    print(f"Recall: {recall:.3f}")
    print(f"F1: {f1:.3f}")
```

# Régularisation L2 (Ridge)

**Problème :** Surapprentissage (overfitting)

**Solution :** Ajouter une pénalité sur les poids élevés

**Fonction de coût régularisée :**

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

**Nouveau gradient :**

$$\nabla J_{\text{reg}} = \nabla J + \frac{\lambda}{m} \boldsymbol{\theta}$$

**Paramètre $\lambda$ :**

- $\lambda = 0$ : Pas de régularisation
- $\lambda = 0.1$ : Faible régularisation
- $\lambda = 1.0$ : Régularisation moyenne
- $\lambda > 10$ : Forte régularisation

# Implémentation Régularisation L2

```
def linear_reg_regularized(X, y, lambda_reg=0.1,
                           alpha=0.01, iterations=1000)
                                  :
    m, n = X.shape
    theta = np.zeros(n)

    for i in range(iterations):
        y_pred = X @ theta

        # Gradient avec r gularisation L2
        gradient = (1/m) * X.T @ (y_pred - y)
        gradient[1:] += (lambda_reg / m) * theta[1:]

        theta = theta - alpha * gradient

    return theta

# Comparer diff rents lambda
for lam in [0, 0.1, 1.0, 10.0]:
```

# Utiliser Scikit-learn

**Au lieu d'implémenter manuellement :**

```
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Diviser les donn es
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Cr er et entra ner le mod le
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

#   valuer
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Exactitude: {accuracy:.3f}")
```

# Pipeline Complet

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Pipeline : pr traitement + mod le
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(C=0.1))
])

# Entra ner
pipeline.fit(X_train, y_train)

# Pr dictions
y_pred = pipeline.predict(X_test)
y_pred_proba = pipeline.predict_proba(X_test)

#   valuation
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

# Validation Croisée (K-Fold)

**Problème :** Un seul ensemble test peut être biaisé
**Solution :** K-Fold Cross-Validation

```
from sklearn.model_selection import cross_val_score

# K-Fold avec K=5
scores = cross_val_score(model, X, y, cv=5)

print(f"Score moyen: {scores.mean():.3f}")
print(f" cart -type: {scores.std():.3f}")
print(f"Scores individuels: {scores}")
```

**Processus :**

1. Diviser en 5 sections
2. Entraîner 5 modèles (chacun sur 4 sections)
3. Évaluer sur la section restante
4. Moyenne des 5 scores

# Sélection des Hyperparamètres

**Hyperparamètres :** paramètres que vous fixez (pas appris)

- Taux d'apprentissage $\alpha$
- Paramètre de régularisation $\lambda$
- Nombre d'itérations

**Grid Search :**

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10],
    'max_iter': [100, 500, 1000]
}

grid = GridSearchCV(LogisticRegression(),
                    param_grid, cv=5)
grid.fit(X_train, y_train)
print(f"Meilleurs params: {grid.best_params_}")
```

# Pièges Courants à Éviter

1. **Fuite de données (Data Leakage)**
   - Normaliser AVANT de diviser
   - Diviser d'abord, normaliser après
2. **Overfitting**
   - Score train 100%, test 50%
   - Utiliser régularisation + validation croisée
3. **Underfit**
   - Score train 50%, test 50%
   - Augmenter la complexité du modèle
4. **Ignorer le déséquilibre de classes**
   - 99% classe A, 1% classe B : utiliser accuracy
   - Utiliser F1 score ou weighted metrics

# Bonnes Pratiques

**À faire :**

- Visualiser les données
- Utiliser validation croisée
- Normaliser les features
- Tester plusieurs modèles
- Documenter le code
- Sauvegarder le meilleur modèle

**À éviter :**

- Utiliser tout le dataset pour l'entraînement
- Oublier de normaliser
- Tester sur les données d'entraînement
- Croire à 100% d'exactitude
- Négliger les valeurs aberrantes

# Ressources Utiles

**Bibliothèques Python essentielles :**

- **NumPy** : calculs mathématiques
- **Pandas** : manipulation de données
- **Scikit-learn** : modèles ML
- **Matplotlib/Seaborn** : visualisation
- **TensorFlow/PyTorch** : deep learning

**Documentation :**

- scikit-learn.org
- kaggle.com (compétitions + datasets)
- github.com (code open-source)

# Résumé

**Concepts clés :**

1. **Régression Linéaire** : prédire des valeurs continues
2. **Classification Logistique** : prédire des classes binaires
3. **Optimisation** : gradient descent, équation normale
4. **Évaluation** : métriques et validation croisée
5. **Régularisation** : éviter le surapprentissage
6. **Implémentation** : Python + scikit-learn

**Point central :** Les mathématiques et la programmation créent des modèles qui apprennent des données.

# Merci !

Questions ?

Sitraka FORLER
Centrale Marseille
Novembre 2025